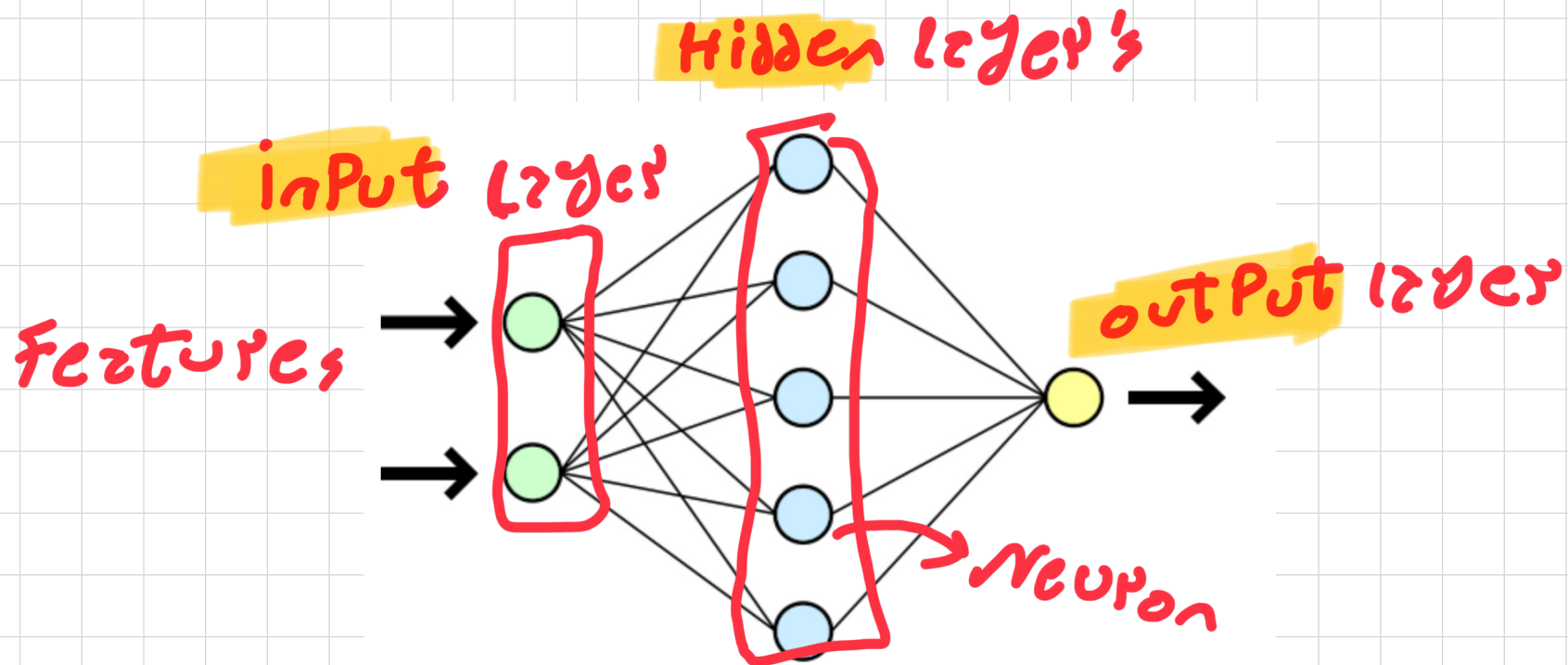
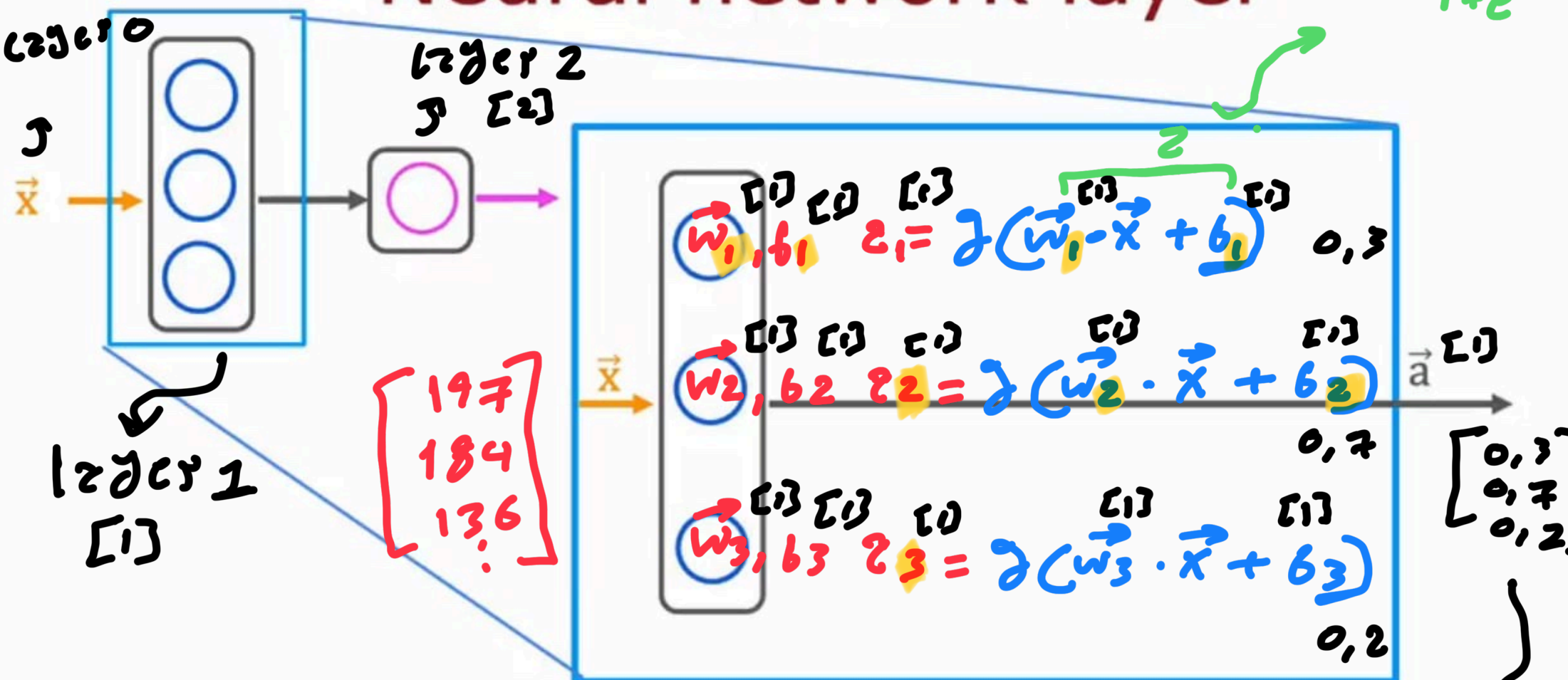


Neural network

- Is a type of ML model inspired by the function of human brain
- Used in image recognition, NLP and predictions

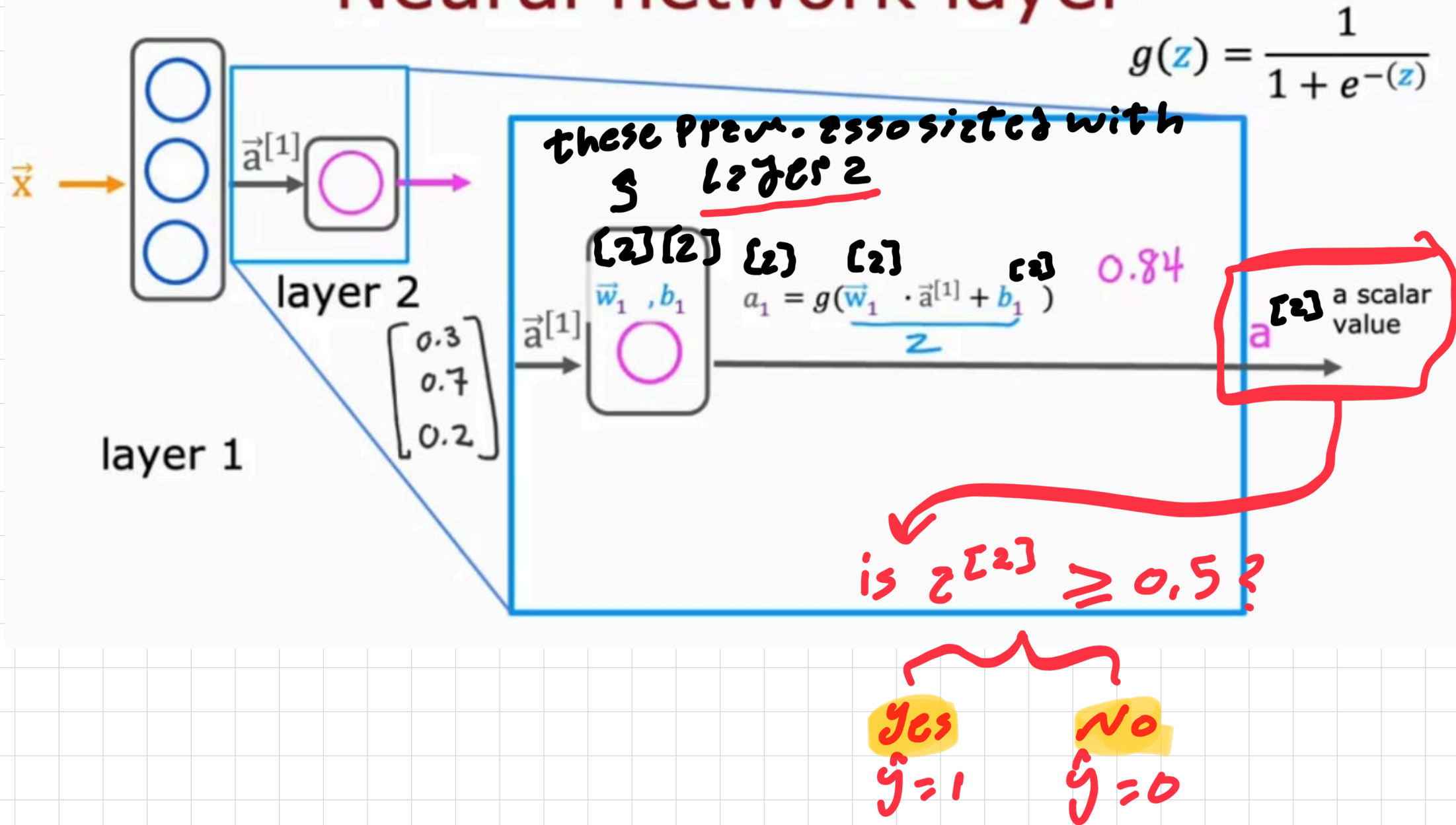


Neural network layer $\sigma(z) = \frac{1}{1+e^{-z}}$

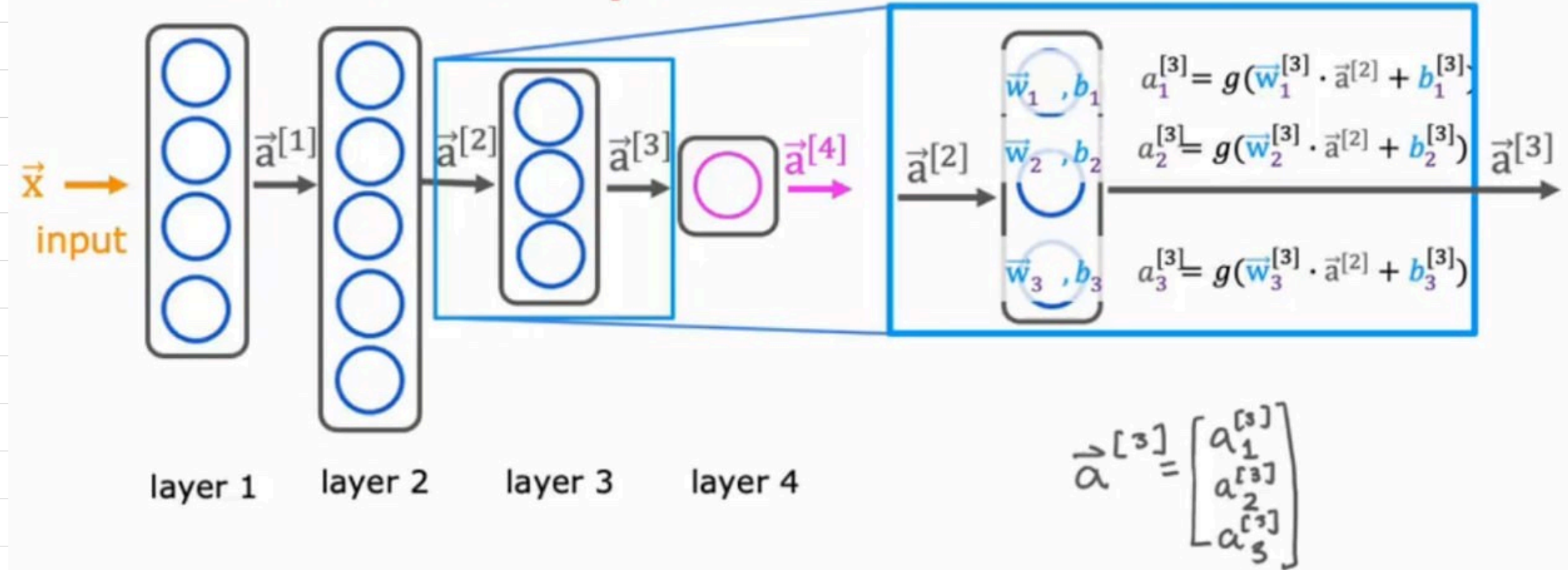


Vector of activation values from layer 1

Neural network layer



More complex neural network



Activation value of layer j

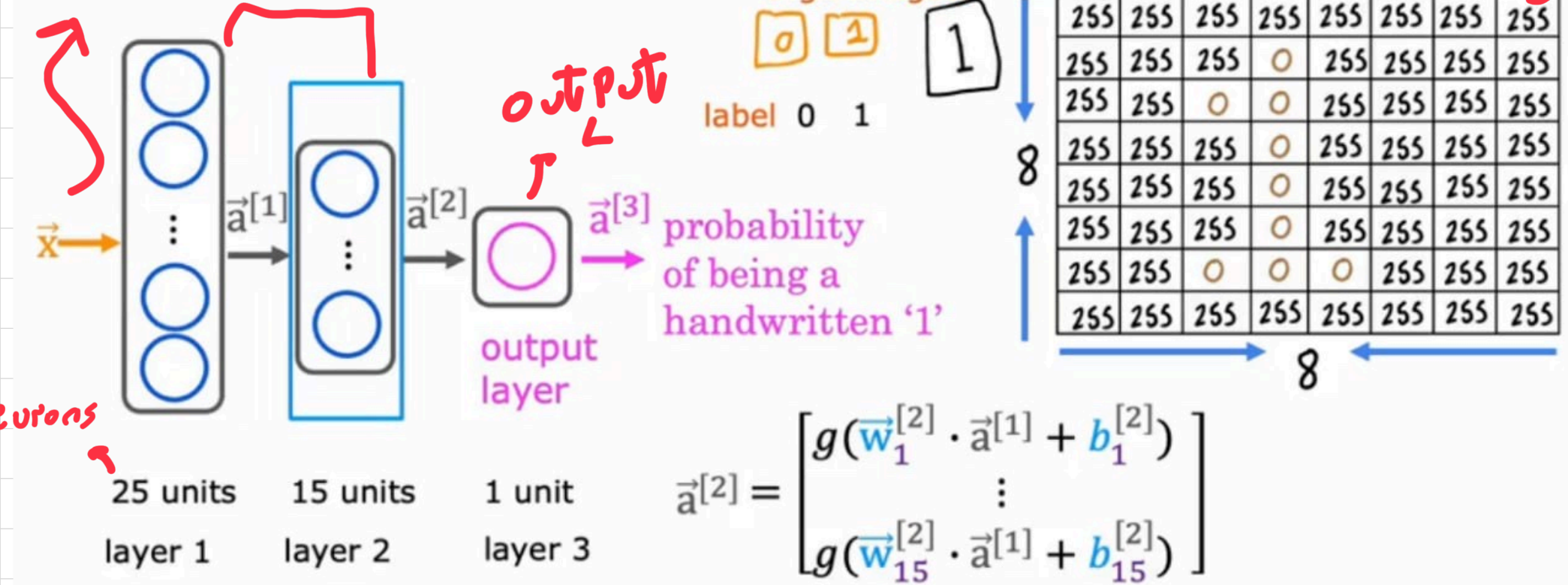
$$z_j^{[L]} = g(\vec{w}_j^{[L]} \cdot z_j^{[L-1]} + b_j^{[L]})$$

output of prev. layer

$L \rightarrow$ layer

$j \rightarrow$ unit (neuron)

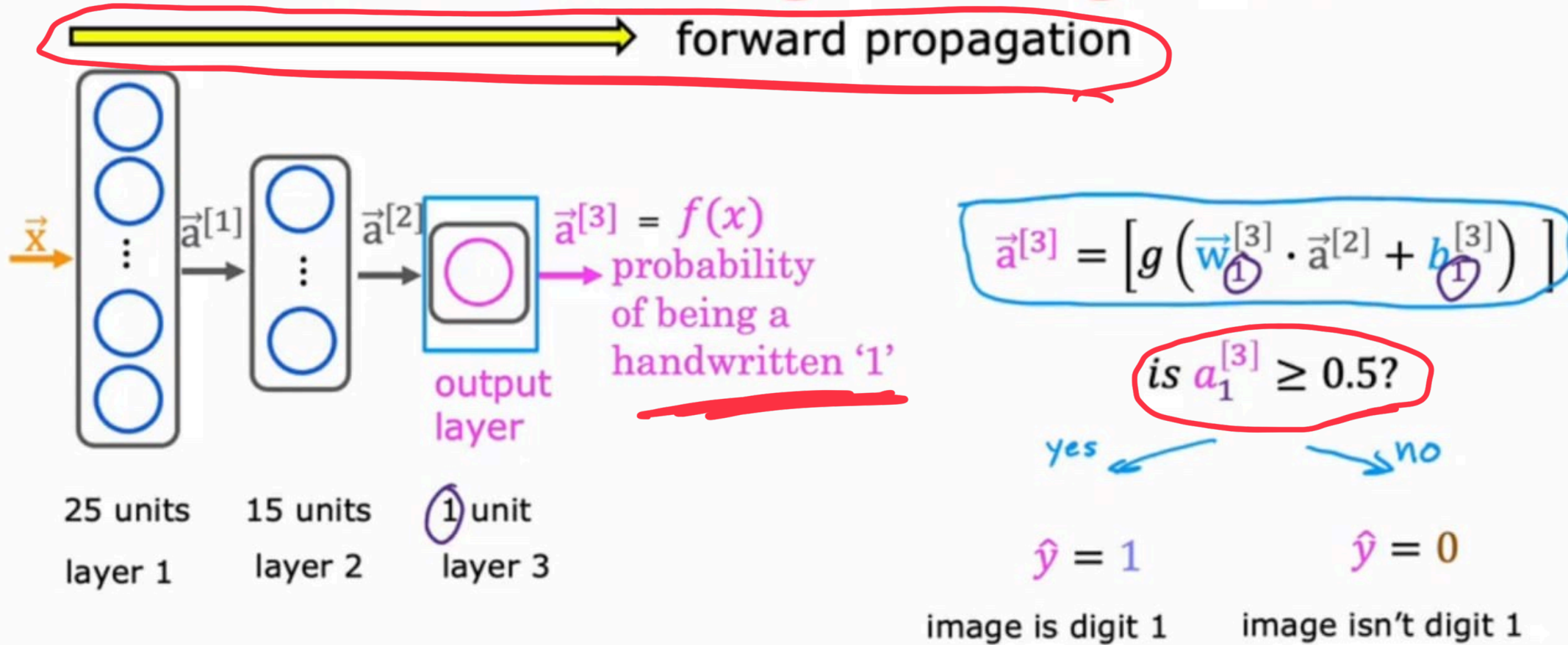
Handwritten digit recognition intensity



Each layer applies $z^L = \vec{w}^{[L]} \cdot \vec{z}^{[L-1]} + b^{[L]}$

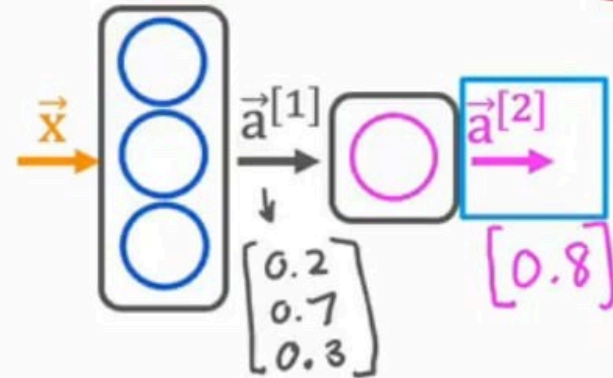
$$\vec{z}^{[1]} = \begin{bmatrix} g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]}) \\ \vdots \\ g(\vec{w}_{25}^{[1]} \cdot \vec{x} + b_{25}^{[1]}) \end{bmatrix} \quad \vec{z}^{[2]} = \begin{bmatrix} g(\vec{w}_1^{[2]} \cdot \vec{z}^{[1]} + b_1^{[2]}) \\ \vdots \\ g(\vec{w}_{15}^{[2]} \cdot \vec{z}^{[1]} + b_{15}^{[2]}) \end{bmatrix}$$

Handwritten digit recognition



Tensor flow

Build the model (Neural Network) using TensorFlow



```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)
```

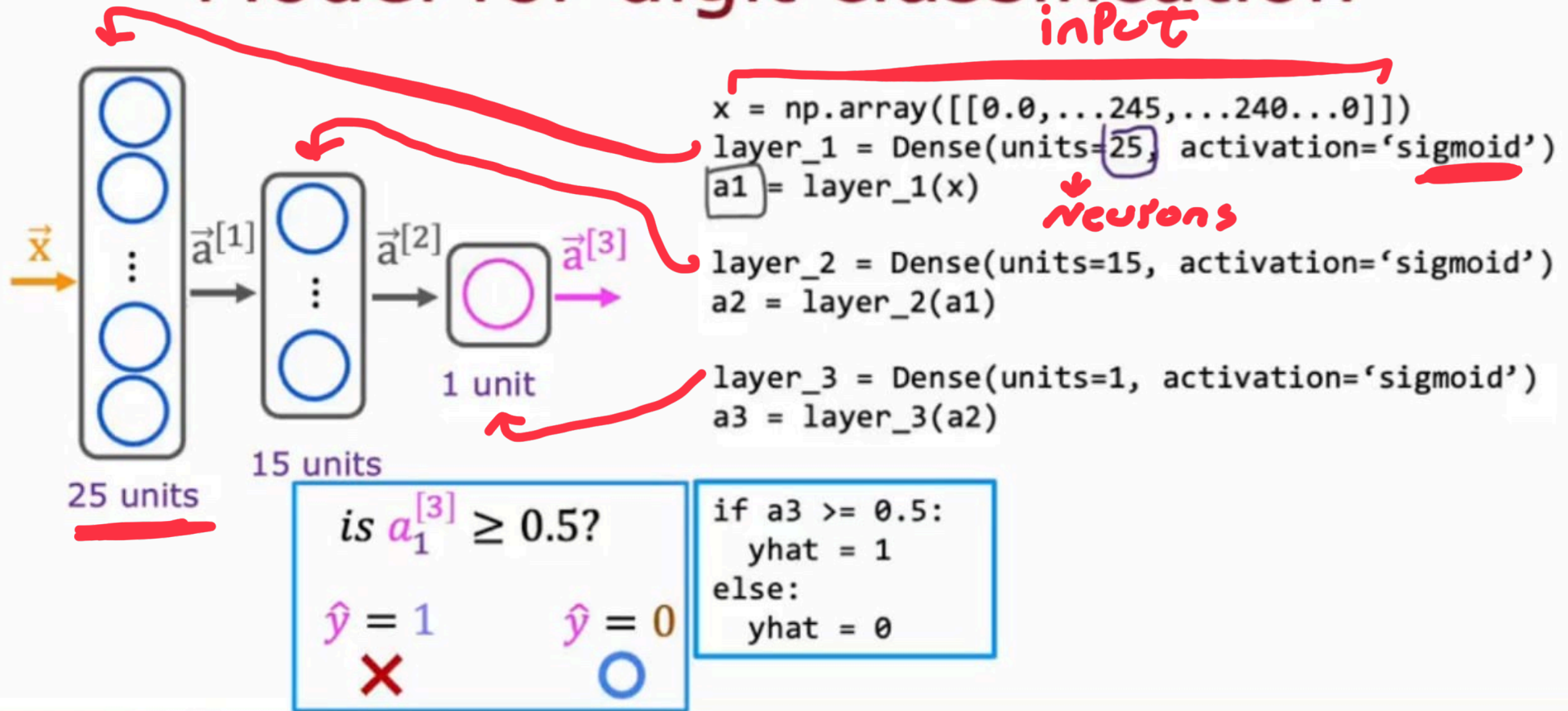
```
layer_2 = Dense(units=1, activation='sigmoid')  
a2 = layer_2(a1)
```

is $a_1^{[2]} \geq 0.5$?
yes $\hat{y} = 1$ $\times$
no $\hat{y} = 0$ $\bigcirc$

```
if a2 >= 0.5:  
    yhat = 1  
else:  
    yhat = 0
```

Digit classification ex

Model for digit classification



Datat in numpy Vs tensorflow

numpy : 2D array

np.array([[1, 2, 3],
[4, 5, 6]])

→ $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

2 × 3

np.array([[0.1, 0.2],
[0.5, 0.8],
[0.3, 0.4]])

→ $\begin{bmatrix} 0.1 & 0.2 \\ 0.5 & 0.8 \\ 0.3 & 0.4 \end{bmatrix}$

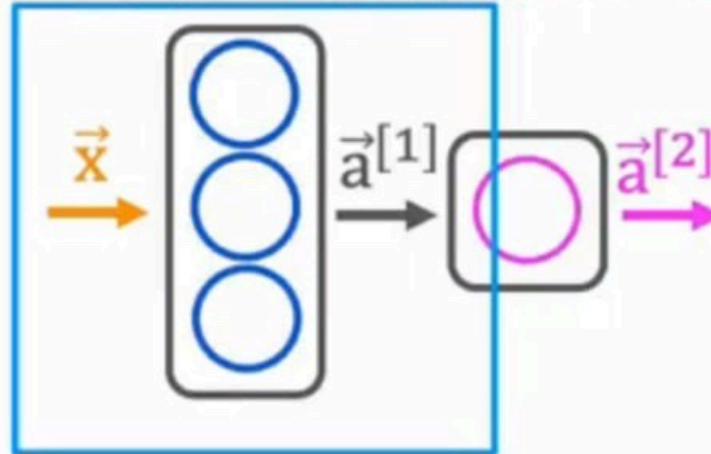
3 × 2

nP.2r2y ([200, 17]) $\rightarrow$ [200 17] 1x2

nP.2r2y ([200,
[5]]) $\rightarrow$ $\begin{bmatrix} 200 \\ 17 \end{bmatrix}$ 2x1

nP.2r2y ([200, 17]) $\rightarrow$ 1D "vector"

Activation vector



```
x = np.array([[200.0, 17.0]])  
layer_1 = Dense(units=3, activation='sigmoid')  
a1 = layer_1(x)
```

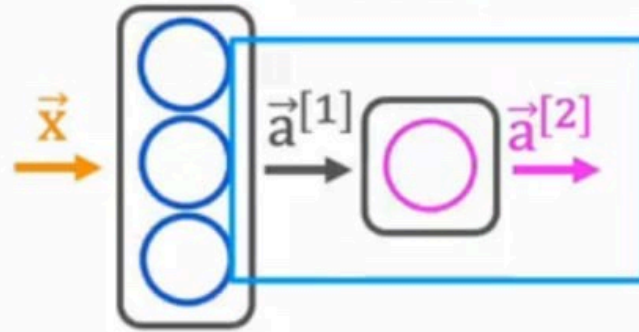
→ $[[0.2, 0.7, 0.3]]$ 1 x 3 matrix

→ tf.Tensor([[0.2 0.7 0.3]], shape=(1, 3), dtype=float32)

```
a1.numpy()
```

```
array([[0.2, 0.7, 0.3]], dtype=float32)
```

Activation vector



```
→ layer_2 = Dense(units=1, activation='sigmoid')  
→ a2 = layer_2(a1)
```

↪ $[[0.8]]$ ←

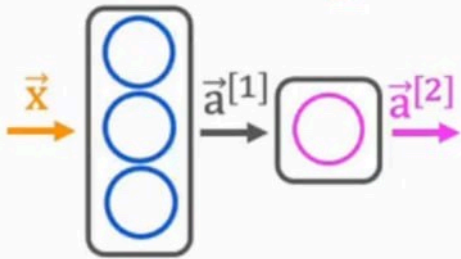
1 x 1 *tensor matrix*

```
→ tf.Tensor([[0.8]], shape=(1, 1), dtype=float32)
```

```
→ a2.numpy()
```

```
array([[0.8]], dtype=float32)
```

Building a neural network architecture



```

→ layer_1 = Dense(units=3, activation="sigmoid")
→ layer_2 = Dense(units=1, activation="sigmoid")
→ model = Sequential([layer_1, layer_2])
    
```

```

x = np.array([[200.0, 17.0],
              [120.0, 5.0],
              [425.0, 20.0],
              [212.0, 18.0]])
    
```

4 x 2

targets `y = np.array([1,0,0,1])`

```

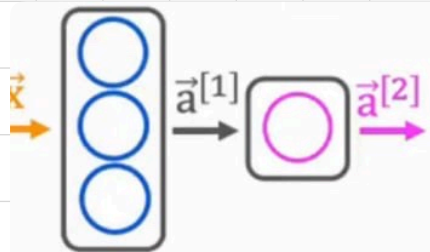
model.compile(...)
model.fit(x,y)
    
```

← more about this next week!

```

→ model.predict(x_new)
    
```

		y
200	17	1
120	5	0
425	20	0
212	18	1



```

→ model = Sequential([
→   Dense(units=3, activation="sigmoid"),
→   Dense(units=1, activation="sigmoid")])
    
```

```

x = np.array([[200.0, 17.0],
              [120.0, 5.0],
              [425.0, 20.0],
              [212.0, 18.0]])
    
```

4 x 2

targets `y = np.array([1,0,0,1])`

```

model.compile(...)
model.fit(x,y)
    
```

← more about this next week!

```

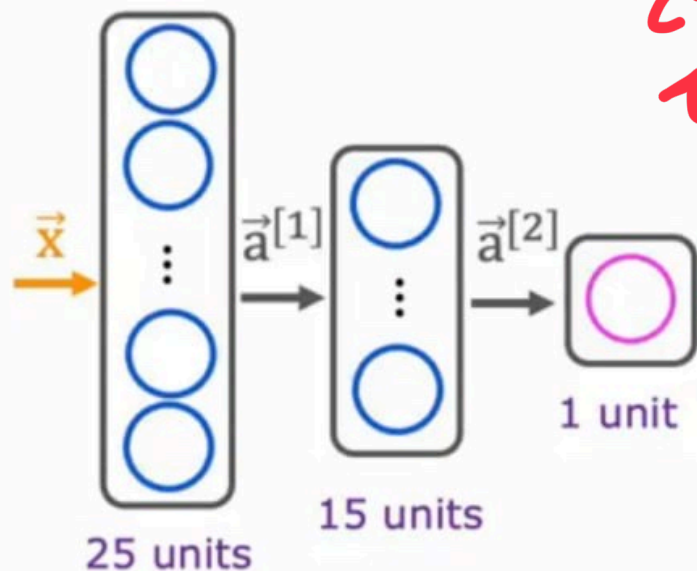
→ model.predict(x_new)
    
```

		y
200	17	1
120	5	0
425	20	0
212	18	1

← simple

Digit classification model

can be simplified



```
→ layer_1 = Dense(units=25, activation="sigmoid")  
→ layer_2 = Dense(units=15, activation="sigmoid")  
→ layer_3 = Dense(units=1, activation="sigmoid")  
→ model = Sequential([layer_1, layer_2, layer_3])  
model.compile(...)
```

```
x = np.array([[0..., 245, ..., 17],  
              [0..., 200, ..., 184]])
```

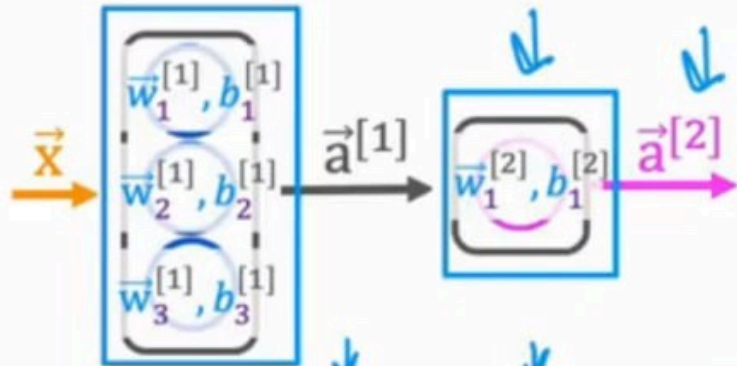
```
y = np.array([1,0])
```

```
model.fit(x,y) ← more about this next week!
```

```
→ model.predict(x_new)
```


Neural Network Implementation in Python

forward prop (coffee roasting model)



$x = \text{np.array}([200, 17])$

$$a_1^{[1]} = g(\bar{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]})$$

$w1_1 = \text{np.array}([1, 2])$

$b1_1 = \text{np.array}([-1])$

$z1_1 = \text{np.dot}(w1_1, x) + b1_1$

$a1_1 = \text{sigmoid}(z1_1)$

1D arrays

$$a_2^{[1]} = g(\bar{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]})$$

$w1_2 = \text{np.array}([-3, 4])$

$b1_2 = \text{np.array}([1])$

$z1_2 = \text{np.dot}(w1_2, x) + b1_2$

$a1_2 = \text{sigmoid}(z1_2)$

$a1 = \text{np.array}([a1_1, a1_2, a1_3])$

$$a_1^{[2]} = g(\bar{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]})$$

$\Rightarrow w2_1 = \text{np.array}([-7, 8, 9])$

$\Rightarrow b2_1 = \text{np.array}([3])$

$\Rightarrow z2_1 = \text{np.dot}(w2_1, a1) + b2_1$

$\Rightarrow a2_1 = \text{sigmoid}(z2_1)$

$w_1^{[2]}$ $w2_1$

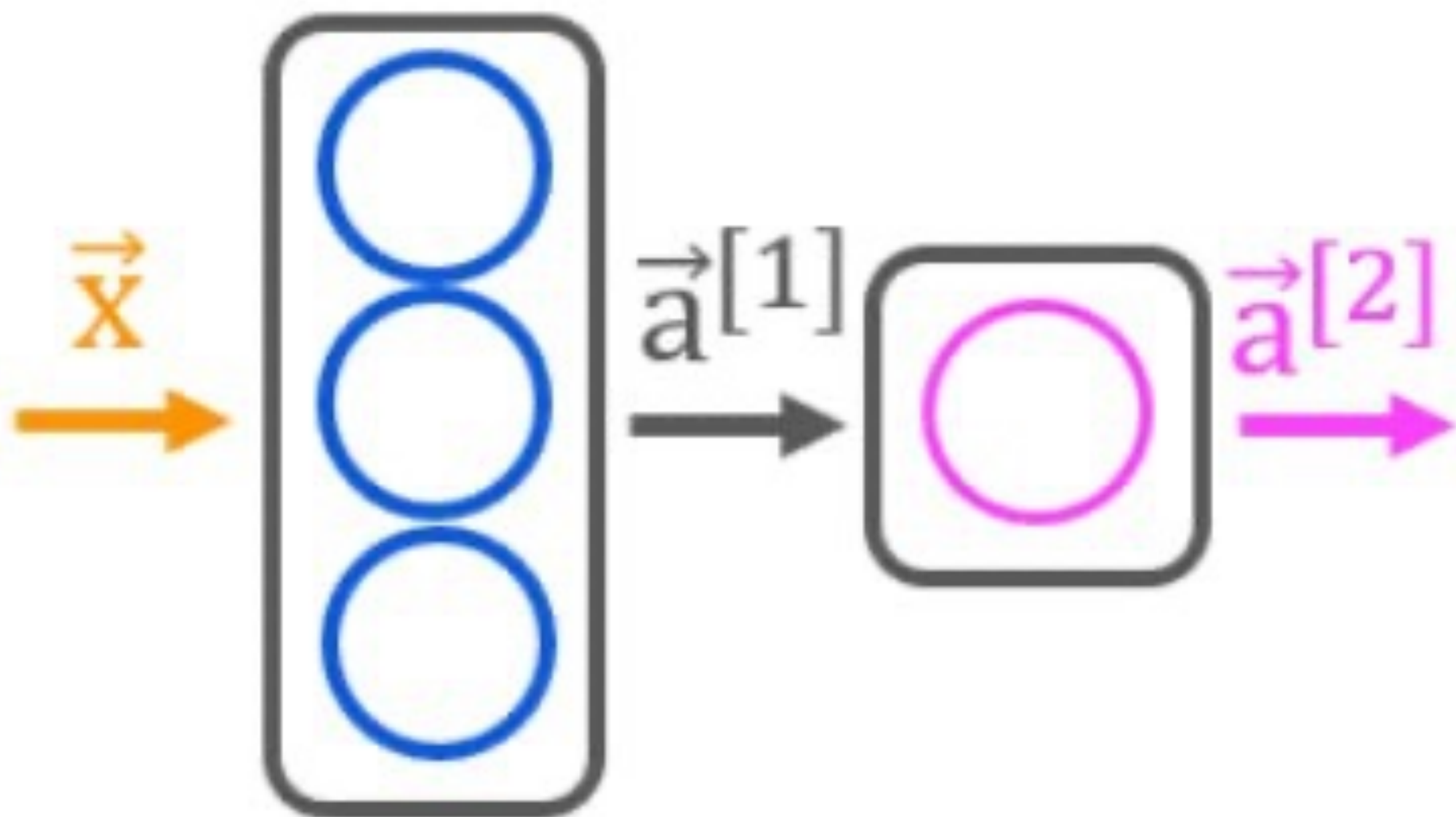
$$a_3^{[1]} = g(\bar{w}_3^{[1]} \cdot \vec{x} + b_3^{[1]})$$

$w1_3 = \text{np.array}([5, -6])$

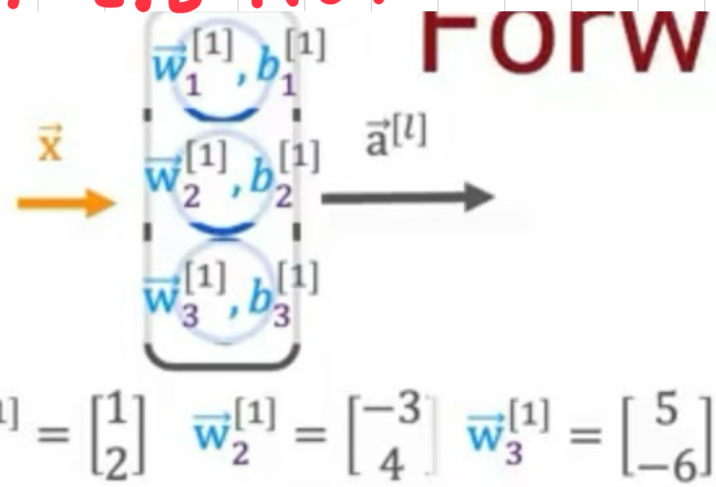
$b1_3 = \text{np.array}([2])$

$z1_3 = \text{np.dot}(w1_3, x) + b1_3$

$a1_3 = \text{sigmoid}(z1_3)$



Forward Prop



Each neurons:

$$z_j = \sigma(w_j \cdot z_{in} + b_j)$$

sigmoid

$x = \text{np.zeros}([200, 17])$

1D array

$w = \text{np.zeros}([2, 3])$
 $\begin{bmatrix} 1, -3, 5 \\ 2, 4, -6 \end{bmatrix}$

2x3 matrix, each column represents w_j^L 2D

$b = \text{np.zeros}([1, 3])$ 1D array

$z_{in} = \text{np.zeros}([2, 4])$ $z^{[0]} = \vec{x}$

def dense (z-in, w, b):

units = w.shape[1]

z-out = np.zeros(units) [0,0,0]

for j in range(units): 0, 1, 2

w = W[:, j] → Pull the col

z = np.dot(w, z-in) + b[j]

z-out[j] = σ(z) sigmoid

return z-out

def sequential(x):

z1 = dense(x, w1, b1) z[1]

z2 = dense(z1, w2, b2) z[2]

z3 = dense(z2, w3, b3) z[3]

z4 = dense(z3, w4, b4) z[4]

f-x = z4

return f-x

for loops

$X = \text{np.zeros}([200, 17])$ 1D

$W = \text{np.zeros}(\begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix})$

$b_1^L = -1, b_2^L = 1, b_3^L = 2$

$b = \text{np.zeros}([-1, 1, 2])$ 1D array

def dense (z-in, W, b):

units = W.shape[1]

z-out = np.zeros(units)

for j in range(units):

w = W[:, j]

z = np.dot(w, z-in) + b[j]

z-out[j] = g(z)

return z-out

Vectorization

$X = \text{np.zeros}([200, 17])$ 2D

$W = \text{np.zeros}(\begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix})$ 2D array

$b = \text{np.zeros}([-1, 1, 2])$ 2D

def dense (z-in, W, b):

units = W.shape[1]

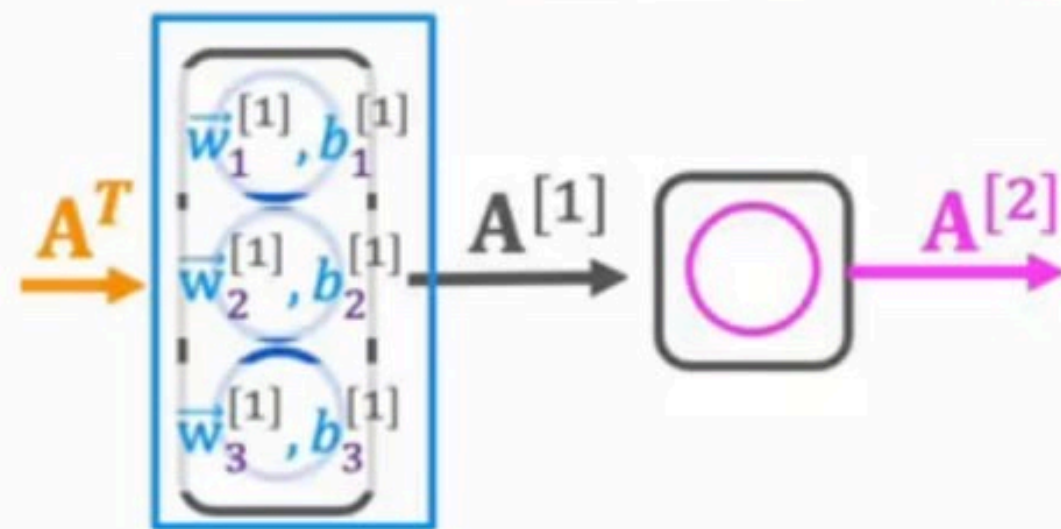
$z = W @ X + b$

z-out = sigmoid(z)

return z-out

Vectorized

Dense layer vectorized



$$A^T = \begin{bmatrix} 200 & 17 \end{bmatrix}$$

1 x 2

$$W = \begin{bmatrix} 1 & -3 & 5 \\ -2 & 4 & -6 \end{bmatrix}$$

2 x 3

$$B = \begin{bmatrix} -1 & 1 & 2 \end{bmatrix}$$

1 x 3

$$Z = A^T W + B$$

$$\begin{bmatrix} 165 & -531 & 900 \end{bmatrix}$$

$z_1^{[1]} \quad z_2^{[1]} \quad z_3^{[1]}$

$$A = g(Z)$$

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

```
A
AT = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([-1, 1, 2])
```

```
def dense(a_in, W, b):
    z = np.matmul(a_in, W) + b
    a_out = g(z)
    return a_out
```

$[[1, 0, 1]]$